TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN



CÔNG NGHỆ PHẦN MỀM

Assignment 1: Hiện thực nghiệp vụ end-to-end theo Scrum

BrewLite – Ứng dụng đặt cà phê

GVHD: Từ Lãng Phiêu

SV: Nguyễn Thành Long – MSSV: .....

Trần Quốc Tài – MSSV: .....

TP. HỒ CHÍ MINH, THÁNG 11/2026

Mục lục

1 Giới thiệu **2**

1.1 Đề tài và mục tiêu 2

1.2 Nhóm và kho mã nguồn 2

1.3 Công nghệ 2

2 Quy trình Scrum **3**

2.1 Kế hoạch sprint 3

2.2 Quy ước làm việc 3

3 Hiện thực 10 task **3**

3.1 Quy tắc nghiệp vụ đã chốt 4

4 Task 10: nghiệp vụ backend **4**

4.1 State machine đơn hàng 5

4.2 Chống gửi trùng và khóa lạc quan 5

4.3 Tồn kho khi đặt đồng thời 6

4.4 Khuyến mãi và điểm thưởng 7

5 API **7**

6 Kiểm thử và kết quả **7**

6.1 Ba tiêu chí chấm của Task 10 8

6.2 Chạy thử luồng thật 8

7 Phân công và bằng chứng đóng góp **9**

8 Hạn chế và hướng phát triển **9**

1 Giới thiệu

1.1 Đề tài và mục tiêu

BrewLite là ứng dụng đặt đồ uống không dùng tiền mặt. Luồng nghiệp vụ tối thiểu theo đề: *xem menu → chọn size/topping → giỏ hàng → đặt đơn → thanh toán → theo dõi đơn*. Assignment 1 chấm **việc hiện thực chạy được từ đầu đến cuối** theo quy trình Agile Scrum: 10 task, mỗi task 1 điểm, công nghệ bắt buộc NestJS (backend) và Next.js (frontend). Các sơ đồ phân tích thiết kế, SRS và kiến trúc thuộc phạm vi Assignment 2 và không đưa vào báo cáo này.

1.2 Nhóm và kho mã nguồn

Thành viên	GitHub	Vai trò	Khu vực mã
Nguyễn Thành Long	`Longcodepro`	Backend, DevOps, Scrum Master	`backend/`, `.github/`, `docker-compose.yml`
Trần Quốc Tài	`TaiTran263`	Frontend, UI/UX	`frontend/`

Kho mã: https://github.com/Longcodepro/Project_SE_Assignment01_HK1_2026_2027. Mỗi bài tập có một kho riêng; kho của Assignment 1 dùng để *kiểm chứng phân công*: mỗi chức năng một nhánh, một Pull Request, một issue, nên có thể truy từ từng task về người thực hiện.

1.3 Công nghệ

Thành phần	Công nghệ
Frontend	Next.js 16 (App Router), TypeScript, Tailwind CSS, Zustand (giỏ hàng, phiên đăng nhập), Axios
Backend	NestJS 12, TypeScript (ESM), class-validator, JWT, `bcryptjs`, Swagger tại `/docs`
Cơ sở dữ liệu	PostgreSQL 16, Prisma 7 (7 migration đến 27/10, thêm 3 migration ở mục 4)
Thanh toán	Cổng thanh toán mô phỏng (Ví / Thẻ) viết trong `payments/mock-gateway.ts`
Kiểm thử	Vitest + Supertest (unit), test tích hợp trên PostgreSQL thật, Postman/newman
DevOps	Docker Compose (3 dịch vụ), GitHub Actions CI/CD, ảnh Docker đẩy lên GHCR

2 Quy trình Scrum

2.1 Kế hoạch sprint

Nhóm chia 3 sprint theo gợi ý của đề. Dữ liệu dưới đây lấy từ issue và milestone trên GitHub (issue #3 đến #19); các issue đều đã đóng.

Sprint	Mục tiêu và issue	Hoàn thành
Sprint 1	Nền tảng và menu: #3 CI, #4 API sản phẩm, #5 trang Menu	23/10 – 25/10/2026
Sprint 2	Giỏ hàng, đặt đơn, đăng nhập: #6 chi tiết sản phẩm, #7 giỏ hàng, #8 API tạo đơn, #9 JWT	26/10/2026
Sprint 3	Thanh toán, bàn giao, nghiệp vụ nâng cao: #10 thanh toán, #11 màn thanh toán, #12 lịch sử đơn + docker, #13 màn lịch sử, #14–#17 Task 10, #19 README	27/10/2026
Ngoài sprint	#18 CD (đóng gói ảnh Docker lên GHCR, cấu hình Cloudflare Tunnel)	27/10/2026

Bảng 1: Issue theo milestone (nguồn: GitHub Issues)

2.2 Quy ước làm việc

- Mỗi chức năng một nhánh `<tên>/task-N-mô-tả`; không đẩy trực tiếp lên `main`.
- Commit dạng `<loại>(task-N): <mô tả>` bằng tiếng Việt, mỗi commit một mục đích.
- Mỗi Pull Request gắn một issue (`Closes/Refs #N`); CI phải xanh mới merge.
- Trước khi commit backend: `npm run lint && npm run build && npm test`.

3 Hiện thực 10 task

Bảng sau đối chiếu từng task của đề với hiện thực. Cột PR là Pull Request đã merge vào `main`.

#	Task của đề	Hiện thực	Người	PR
1	Khởi tạo dự án	Hai thư mục `frontend`, `backend`, Docker, README, `.env.example`	Tài, Long	#1, #2
2	API danh sách sản phẩm	`GET /products` (Prisma, 6 món mẫu), Swagger, Postman	Long	#24, #28
3	Trang Menu	Lưới sản phẩm; trạng thái loading, rỗng, lỗi + thử lại	Tài	#22
4	Chi tiết, tùy chọn	Chọn size S/M/L, topping; giá cập nhật theo tùy chọn	Tài	#30
5	Giỏ hàng	Thêm, sửa, xóa; Zustand lưu `localStorage`; badge số lượng	Tài	#34
6	API tạo đơn	`POST /orders`, validate, đơn `PENDING`, tổng tiền do server tính	Long	#31
7	Đăng ký, đăng nhập	`POST /auth/register`, `/auth/login`, bcrypt, JWT, guard; giao diện đăng nhập	Long, Tài	#32, #46
8	Thanh toán không tiền mặt	`POST /payments` (Ví/Thẻ); màn thanh toán và xác nhận	Long, Tài	#37, #35
9	Xác nhận, lịch sử, bàn giao	`GET /orders/me`, màn lịch sử, Docker Compose 3 dịch vụ, README	Long, Tài	#38, #36, #43
10	Nghiệp vụ backend	Xem mục 4	Long	#39–#42, #56–#58

Bảng 2: Đối chiếu task và hiện thực

3.1 Quy tắc nghiệp vụ đã chốt

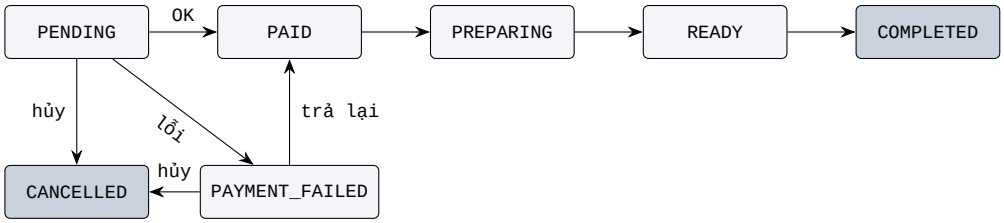
- **Tổng tiền do server tính**: $\sum$ quantity $\times$ (giá + phụ thu size + giá topping). Size S/M/L: $+0$/$+5.000$/$+10.000$đ. Topping: trân châu 8.000, kem cheese 10.000, thạch cà phê 6.000đ (`backend/src/orders/pricing.ts`). Client gửi giá lên bị từ chối (400).
- **Thanh toán mô phỏng**: thẻ có 4 số cuối `0000` bị từ chối; Ví từ chối khi đơn trên 500.000đ.
- Mật khẩu băm `bcryptjs`; không có giá trị mặc định cho `JWT_SECRET`.

4 Task 10: nghiệp vụ backend

Đề yêu cầu bốn ý và ba tiêu chí chấm: (a) chặn chuyển trạng thái sai, (b) gửi hai lần cùng `Idempotency-Key` chỉ tạo một đơn, (c) đặt đồng thời không vượt tồn kho. Sau khi soát lại mã nguồn ngày 14/11/2026, nhóm nhận thấy bản hiện thực ngày 27/10 chưa đủ: máy trạng thái chỉ có 3 trạng thái, chưa có tồn kho và mã giảm giá. Nhóm đã bổ sung ba Pull Request (#56, #57, #58) và chạy lại toàn bộ kiểm thử.

Ý của đề	Hiện thực	PR
1. State machine	7 trạng thái, 8 chuyển hợp lệ, `assertTransition` trả 409 nếu sai	#39, #57
2. Idempotency	Header `Idempotency-Key` cho `POST /orders` và `POST /payments`, ràng buộc duy nhất ở DB	#40
3. Đồng thời / tồn kho	Khóa lạc quan `version` cho đơn; cột `Product.stock`, trừ kho nguyên tử	#41, #56
4. Khuyến mãi, điểm thưởng	Mã giảm giá `BREW10`, `GIAM15K`; tích 1 điểm / 1.000đ	#42, #58

4.1 State machine đơn hàng



Hình 1: Vòng đời đơn (nguồn: `backend/src/orders/order-status.ts`). Ba bước PAID → COMPLETED do nhân viên thực hiện.

```
export const ORDER_TRANSITIONS: Record<OrderStatus, readonly OrderStatus[]> = {
  PENDING: ['PAID', 'PAYMENT_FAILED', 'CANCELLED'],
  PAYMENT_FAILED: ['PAID', 'CANCELLED'],
  PAID: ['PREPARING'],
  PREPARING: ['READY'],
  READY: ['COMPLETED'],
  COMPLETED: [],
  CANCELLED: [],
};
```

Chỉ nhân viên (cột `User.role = STAFF`) mới gọi được `PATCH /orders/:id/status` và `GET /orders/queue`; khách gọi nhận 403. Vai trò được đọc từ cơ sở dữ liệu ở mỗi lần gọi để việc hạ quyền có hiệu lực ngay. Đơn đã `PAID` không hủy được (hoàn tiền nằm ngoài phạm vi). Mọi lần đổi trạng thái đều qua `assertTransition` và khóa lạc quan.

4.2 Chống gửi trùng và khóa lạc quan

Khóa duy nhất `@@unique([userId, idempotencyKey])` (đơn) và `@@unique([orderId, idempotencyKey])` (thanh toán) làm cho yêu cầu lặp trả lại kết quả cũ. Hai yêu cầu cùng khóa đến cùng lúc: bên thua bắt lỗi `P2002` và trả kết quả của bên thắng. Cột `Order.version` tăng

1 sau mỗi lần ghi; câu lệnh cập nhật có điều kiện `where: { id, version }`, không khớp thì 409.

4.3 Tồn kho khi đặt đồng thời

```
57    // Cộng dồn số ly theo sản phẩm (một sản phẩm có thể nằm ở nhiều dòng khác size)
58    const qtyById = new Map<number, number>();
59    for (const i of items) qtyById.set(i.productId, (qtyById.get(i.productId) ?? 0) +
      ↪  i.quantity);
60
61    try {
62      return await this.prisma.$transaction(async (tx) => {
63        // Trừ kho có điều kiện: UPDATE ... WHERE stock >= qty là một câu lệnh nguyên
          ↪  tử,
64        // hai yêu cầu đồng thời không thể cùng thấy "còn hàng" rồi cùng trừ.
65        // Một sản phẩm thiếu hàng thì ném lỗi, transaction hoàn lại các lần trừ
          ↪  trước.
66        for (const [productId, qty] of qtyById) {
67          const { count } = await tx.product.updateMany({
68            where: { id: productId, stock: { gte: qty } },
69            data: { stock: { decrement: qty } },
70          });
71          if (count === 0) {
72            throw new ConflictException(`Sản phẩm ${productId} không đủ hàng cho
              ↪  ${qty} ly`);
73          }
74        }
75        return tx.order.create({
76          data: {
77            id: `ord_${randomBytes(5).toString('hex')}`,
78            userId,
79            idempotencyKey,
80            total,
81            promoCode: promo?.code,
82            discount: promo?.discount ?? 0,
83            items: { create: items },
84          },
85          select: SUMMARY,
86        });
87      });
```

Phép trừ `UPDATE ... WHERE stock >= số ly` là một câu lệnh nguyên tử, nên hai giao dịch đồng thời không thể cùng thấy "còn hàng". Hết hàng thì ném 409 và giao dịch hoàn lại các món đã trừ. Hủy đơn hoàn lại kho trong cùng giao dịch. Migration thêm `CHECK ("stock" >= 0)` làm chốt chặn cuối ở cơ sở dữ liệu.

4.4 Khuyến mãi và điểm thưởng

`POST /orders` nhận `promoCode` tùy chọn (không phân biệt hoa thường). `BREW10` giảm 10%, tối đa 20.000đ; `GIAM15K` giảm 15.000đ cho đơn từ 50.000đ. Mã sai hoặc đơn chưa đủ giá trị trả 400 và không trừ kho. `total` là số phải trả sau giảm nên thanh toán và điểm thưởng tự tính trên số đã giảm. Điểm thưởng: 1 điểm / 1.000đ, làm tròn xuống, cộng trong cùng giao dịch với việc chuyển đơn sang `PAID`; thanh toán lỗi hoặc gửi lại cùng khóa thì không cộng thêm.

5 API

Phương thức	Đường dẫn	Mô tả	Quyền
GET	`/`, `/docs`	Kiểm tra backend; Swagger	công khai
GET	`/products`	Menu, kèm `stock`	công khai
POST	`/auth/register`, `/auth/login`	Đăng ký, đăng nhập (trả `accessToken`)	công khai
POST	`/orders`	Đặt đơn; `Idempotency-Key`, `promoCode` tùy chọn	khách
GET	`/orders/me`	Lịch sử đơn, mới nhất trước	khách
POST	`/orders/:id/cancel`	Hủy đơn chưa thanh toán, hoàn kho	khách
POST	`/payments`	Thanh toán Ví/Thẻ; `Idempotency-Key` tùy chọn	khách
GET	`/orders/queue`	Hàng đợi pha chế	nhân viên
PATCH	`/orders/:id/status`	`PREPARING` → `READY` → `COMPLETED`	nhân viên

6 Kiểm thử và kết quả

Số liệu dưới đây lấy từ lần chạy ngày 14/11/2026 trên nhánh `main` sau khi merge PR #58.

Hạng mục	Nội dung	Kết quả
Lint, build backend	`oxlint`, `nest build`	không lỗi
Unit test	Vitest, 9 file (auth, orders, payments, products, swagger, state machine, mã giảm giá, loyalty, health)	93/93 qua
Test trên PostgreSQL thật	3 test tồn kho + 1 test health (`npm run test:e2e`)	4/4 qua
Postman/newman	4 request, 12 assertion (health, products, 404, Swagger)	12/12 qua
Độ phủ (v8), toàn backend	dòng / nhánh / hàm	83,2% / 75,4% / 88,4%
Độ phủ, `orders`, `payments`	dòng	97,1%, 96,2%
CI GitHub Actions	job `backend`, `backend-db`, `frontend`, `docker`	xanh ở PR #56, #57, #58
CD	đóng gói ảnh Docker backend lên GHCR khi merge vào `main`	xanh

6.1 Ba tiêu chí chấm của Task 10

(a) **Chặn chuyển trạng thái sai**: `order-status.spec.ts` duyệt đủ $7 \times 7 = 49$ cặp trạng thái, chỉ 8 cặp hợp lệ; 8 trường hợp cấm cụ thể (ví dụ `PAID`→`CANCELLED`, `PAID`→`READY`) trả 409.

(b) **Gửi hai lần cùng `Idempotency-Key` chỉ tạo một đơn**: các test `POST /orders` với `Idempotency-Key` (gửi lại, hai yêu cầu cùng lúc, sai định dạng) và test thanh toán gửi lại không cộng điểm lần nữa.

(c) **Đặt đồng thời không vượt tồn kho**: `backend/test/ton-kho.e2e-spec.ts` chạy trên PostgreSQL thật: 20 khách cùng đặt 1 ly khi kho còn 5 $\Rightarrow$ đúng 5 đơn thành công, 15 yêu cầu nhận 409, kho về 0, tổng số ly đã bán bằng 5. Một test khác chứng minh đơn có một món thiếu hàng thì không trừ kho món nào, và cơ sở dữ liệu từ chối `stock = -1`. CI chạy các test này ở job `backend-db`.

6.2 Chạy thử luồng thật

Backend chạy thật trên PostgreSQL tạm (cổng riêng), gọi bằng `curl`. Kết quả rút gọn:

```
Đặt 2 ly M + mã brew10   -> 201 PENDING, total 61.200, discount 6.800, promoCode BREW10
Đặt 2 ly khi kho còn 1    -> 409 "Sản phẩm 1 không đủ hàng cho 2 ly"
Mã giảm giá sai           -> 400 "Mã giảm giá SAI không hợp lệ"
Thanh toán ví             -> 201 PAID, pointsEarned 61
Khách gọi API nhân viên   -> 403
```

```
Nhân viên PAID->COMPLETED -> 409 "Không thể chuyển đơn từ PAID sang COMPLETED"
Nhân viên PREPARING, READY, COMPLETED -> 200 lần lượt
Hủy đơn đã COMPLETED      -> 409
Đặt 1 ly rồi hủy          -> kho 0 -> 1 (hoàn kho)
Thanh toán đơn đã hủy     -> 409 "Đơn đang ở trạng thái CANCELLED, không thể thanh toán"
```

7 Phân công và bằng chứng đóng góp

Số liệu lấy từ `git` và GitHub ngày 14/11/2026, tính đến trước 3 Pull Request bổ sung #56–#58 (đều của Long).

Nội dung	Cách tính	Long	Tài
Pull Request đã merge	theo tiền tố nhánh `long/`, `tai/`; PR khởi tạo #1, #2 tính theo người đặt tên nhánh	26	10
Issue được giao	#3 đến #19 (17 issue); #19 giao cho cả hai	11	5
Commit không tính merge	`git shortlog -sn --no-merges`, theo tác giả git	43	8
Dòng mã thêm / xóa	`git log --numstat` trong `backend/`, `frontend/`	+18.600 / −886	+11.686 / −324

Bảng 3: Bằng chứng đóng góp theo từng thành viên

Long phụ trách toàn bộ backend: khởi tạo NestJS, API sản phẩm, đơn hàng, xác thực JWT, thanh toán, lịch sử đơn, bốn phần của Task 10 cùng phần bổ sung tồn kho, mã giảm giá, vòng đời đơn; CI/CD, Docker Compose, README, Swagger/Postman. **Tài** phụ trách frontend: khởi tạo Next.js, trang Menu, chi tiết sản phẩm, giỏ hàng, màn thanh toán và xác nhận, lịch sử đơn.

> **Lưu ý để đọc số liệu đúng.** Một số Pull Request nhánh `tai/` (#46 giao diện đăng nhập, #48 responsive di động, #52 cấu hình URL backend động) có commit mang tài khoản git của Long, nên số commit theo tác giả của Tài thấp hơn khối lượng frontend thực tế. Vì vậy báo cáo trình bày nhiều cách tính (theo nhánh, theo issue, theo tác giả) và không quy về một tỷ lệ phần trăm.

8 Hạn chế và hướng phát triển

- **Frontend chưa dùng các API bổ sung ngày 14/11**: nút hủy đơn, ô nhập mã giảm giá, màn hình hàng đợi cho nhân viên, hiển thị tồn kho. Các API này đã chạy và có test ở backend; phần giao diện thuộc phạm vi của Tài.

- Lịch sử đơn của frontend khi lỗi mạng tự chuyển sang dữ liệu mẫu, chưa phân biệt rõ với dữ liệu thật.
- Mã giảm giá cố định trong mã nguồn (`promo.ts`), chưa có bảng, hạn dùng hay số lượt dùng.
- Tồn kho tính theo ly sản phẩm, chưa theo nguyên liệu; hoàn tiền cho đơn đã thanh toán nằm ngoài phạm vi.
- Vai trò nhân viên cấp bằng cách sửa cột `role` trong cơ sở dữ liệu, chưa có giao diện quản trị.

Các nội dung phân tích thiết kế (SRS, use-case, activity/sequence/class, kiến trúc, ADR, triển khai production) được làm ở Assignment 2.

Tài liệu

[1] NestJS Documentation, https://docs.nestjs.com.

[2] Prisma ORM Documentation, https://www.prisma.io/docs.

[3] Next.js Documentation, https://nextjs.org/docs.

[4] IETF, *The Idempotency-Key HTTP Header Field* (bản thảo), https://datatracker.ietf.org/doc/draft-ietf-httpapi-idempotency-key-header/.

[5] Đề bài lớn *BrewLite – Công nghệ phần mềm*, Trường Đại học Sài Gòn, 2026.